

Inyección de argumentos y opciones

Objetivos de aprendizaje

Determinar cuándo una entrada crea uno o varios elementos de argv, diferenciar opción de operando y explotar o descartar capacidades del programa sin atribuir las a una shell inexistente.

Prerrequisitos

Procesos, argv, shell splitting, parsers de opciones, -- y documentación de herramientas.

Fundamentos técnicos

Un espacio no crea por sí mismo otro argumento. El componente que construye argv decide:

```
subprocess.run(["curl", user_input])          # un solo argumento
subprocess.run(["curl", *shlex.split(user_input)]) # varios argumentos
subprocess.run("curl " + user_input, shell=True)  # shell: splitting y operadores
```

El primer caso puede permitir controlar un operando completo, pero el espacio sigue dentro de él. El segundo habilita argumentos u opciones adicionales sin gramática de shell. El tercero puede convertirse además en command injection.

Modelo mental

```
entrada -> ¿split explícito? -> argv[] -> parser de opciones -> operación
        -> ¿shell? -----> palabras/operadores -> proceso(s)
```

Superficie de ataque

Wrappers de curl, wget, compresores, convertidores, multimedia, compiladores y cualquier función que exponga opciones de una CLI.

Cómo identificarla

- Error unknown option o cambio inocuo de comportamiento propio de la herramienta.
- ; id es literal, pero una opción adicional reconocida cambia la salida.
- Logs muestran más elementos de argv que el previsto.
- Un operando adicional provoca una segunda operación sin segundo proceso.

Preguntas que debo hacerme

1. ¿Quién divide el texto y con qué reglas?
2. ¿Control un elemento, varios o una posición fija?
3. ¿El programa acepta opciones después de operandos?
4. ¿Implementa -- como fin de opciones?
5. ¿Qué capacidad inocua confirma llegada al parser de opciones?

Prueba mínima

Usar una opción no destructiva documentada para la versión detectada y verificar el argv resultante. Un error de opción confirma parser de opciones, no ejecución de código.

Construcción progresiva del payload

1. Entrada válida y salida normal.
2. Un valor con espacio y log argv.
3. Opción inocua, por ejemplo cambiar verbosidad o cabeceras, solo si se crea un elemento adicional.

4. Operando adicional hacia un listener propio.
5. Comparar con -- y con un único argumento literal.

Anatomía de los payloads

- Contexto de entrada: campo resources de un importador.
- Sintaxis original: `subprocess.run(["curl", *shlex.split(resources)])`.
- Entrada controlada: lista textual dividida por `shlex.split`.
- Transformaciones conocidas: JSON decode y splitting POSIX.
- Parser final: parser de opciones de curl.
- Sink: una o varias transferencias de curl.
- Primitiva: añadir un segundo operando HTTP controlado.
- Payload mínimo: `https://allowed.lab/a http://listener.lab/token`.
- Significado de cada componente: primer operando satisface la función; segundo solicita el listener.
- Resultado esperado: dos solicitudes dentro del mismo proceso.
- Control negativo: el mismo texto pasado como un único `argv[1]`.
- Restricción observada: esquemas locales están bloqueados.
- Por qué falla la variante básica: `;` `id` no es opción ni operando válido para una shell ausente.
- Hipótesis de adaptación: abusar de operandos soportados por curl.
- Payload adaptado: segundo URL HTTP del laboratorio.
- Por qué debería funcionar: el wrapper lo convierte en otro elemento y curl admite múltiples URL.
- Evidencia: `argv`, dos callbacks y un solo PID de curl.
- Cuándo no funcionaría: un solo elemento, allowlist por cada URL o wrapper que inserta -- y limita operandos.

Variaciones según el contexto

Construcción	Espacio	;	Riesgo principal
[programa, entrada]	parte del mismo argumento	literal	operando controlado
[programa, *split(entrada)]	crea argumentos	literal	option/argument injection
shell=True con cadena	shell splitting	operador	command injection y argumentos
glob de shell *	nombres se vuelven argumentos	depende del nombre	option injection por filenames

Filtros y bypasses

Bloquear `;` no corrige inyección de opciones. La defensa debe controlar la construcción de `argv`, validar cada operando, insertar -- cuando la herramienta lo soporte y evitar parsers de texto intermedios.

Evidencias de confirmación

`argv` adicional o efecto documentado de una opción/operando en el mismo proceso. Una firma de salida parecida a curl solo identifica una hipótesis de herramienta.

Escalado de impacto

Estudiar únicamente capacidades documentadas de la versión, comenzar con opciones de diagnóstico y demostrar el impacto mínimo. No asumir que todas las herramientas o compilaciones soportan los mismos protocolos/hooks.

Errores frecuentes

- Afirmar que un espacio siempre divide.
- Confundir segundo URL de curl con segundo comando.
- Usar opciones de otra versión.
- Suponer que -- es universal o que el wrapper lo coloca correctamente.
- Saltar de error de opción a ejecución arbitraria.

Diagnóstico de payloads fallidos

Síntoma	Hipótesis	Prueba
espacio aparece escapado en un error	un único argumento	log argv y longitud
; id literal, opción cambia salida	parser de opciones sin shell	comparar PID y argv
opción desconocida	versión/herramienta distinta	obtener versión y manual oficial
-- detiene la opción	parser implementa fin de opciones	comparar antes/después de --
segundo URL no se solicita	un solo argumento o política por URL	log argv y listener por operando
aparecen dos procesos	shell/wrapper adicional	árbol de procesos y comando final

Mitigaciones

Construir argv explícito, un elemento por valor, evitar split de entrada, usar -- cuando esté documentado, allowlist de opciones y operandos, API nativa y privilegio mínimo.

Relación con pentesting y certificaciones

Se evalúa la capacidad de dibujar argv y atribuir el efecto al parser correcto.

Caso guiado

Caso A — Básico: dos recursos en un importador

El log muestra argv=["curl","https://allowed/a","http://listener/t1"]; llegan dos peticiones y existe un solo PID.

Observación: la entrada creó dos operandos. Qué sé: el wrapper divide y curl procesa ambas URL. Hipótesis: argument injection o shell. Experimento: incluir ; como token y revisar árbol de procesos. Resultado: ; es un operando inválido y no nace otro proceso. Conclusión: inyección de argumentos, no de comandos. Siguiente paso: revisar capacidades documentadas y política por operando.

Caso de adaptación

Caso B — El espacio permanece dentro del argumento

El mismo payload en otra versión produce argv=["curl","https://allowed/a http://listener/t1"] y URL rejected: Malformed input.

Observación: no hay segundo elemento. Qué sé: un espacio no fue dividido. Hipótesis: lista directa o quoting del wrapper. Experimento: log de longitud y opción inocua. Resultado: siempre dos elementos totales: programa y valor. Conclusión: la variante básica falla porque no existe splitting; no hay adaptación legítima con otro separador sin nueva

evidencia. Siguiendo paso: descartar esta vía.

Caso C — Transferencia: perfil de conversión

Un conversor recibe `profile="-loglevel debug"`; el log muestra que el wrapper aplica `shlex.split(profile)` y la herramienta cambia verbosidad. No se revela la técnica en el título.

Resolución: texto -> split -> opciones de la herramienta. El control pasa el mismo texto como un elemento y recibe "unrecognized option". La primitiva transferida es controlar opciones, no ejecutar una shell.

Caso D — Falso positivo: medidor de progreso

La respuesta contiene % Total y % Received, pero ninguna opción inocua altera el comportamiento y el backend usa una biblioteca que copia ese formato.

Observación: firma compatible con curl. Hipótesis: CLI curl o biblioteca. Experimento: árbol de procesos, error de opción y versión. Resultado: no hay proceso curl ni parser de opciones. Conclusión: la firma no confirma argument injection.

Comparación: Command Injection vs Argument Injection

Experimento	Shell vulnerable	argv con splitting	argv único
<code>; printf MARK</code>	ejecuta MARK	<code>;/printf</code> son argumentos	texto literal único
opción inocua documentada	puede llegar tras splitting	cambia herramienta	queda dentro del operando
árbol de procesos	puede mostrar shell y segundo proceso	un proceso objetivo	un proceso objetivo
log argv	resultado tras expansión	varios elementos controlados	un elemento controlado

Ejercicios

1. Dibuja argv para las tres llamadas Python de Fundamentos.
2. Explica por qué dos callbacks de curl no implican dos comandos.
3. Diseña un control con `--` sin asumir que la herramienta lo soporta.
4. Distingue option injection de un operando adicional.

Resumen

La inyección de argumentos depende de quién crea argv. Un espacio solo divide cuando una capa lo interpreta; el efecto pertenece al parser de opciones del programa, no a una shell por defecto.

Chuleta operativa

1. Código o log de construcción.
2. Número y contenido de argv.
3. Shell sí/no.
4. Opción inocua y versión.
5. `--` documentado.
6. PID y efecto mínimo.

Referencias

- [Python subprocess](#)
- [curl manual](#)

- [GNU Bash Reference Manual](#)
- [Concepto interno](#) (argument-injection)

Navegación

Anterior: [Command injection](#). Índice: [curso](#). Práctica: [cuaderno](#).